

Capítulo 6

Encapsulamento



Fonte: Gamma – IA | Adaptada

6.1 Definição e Benefícios



O encapsulamento é um dos pilares essenciais da orientação a objetos, responsável por proteger os dados de um objeto contra acessos e alterações inadequadas. Ele permite controlar o acesso aos atributos de uma classe por meio de modificadores de acesso e métodos.

No universo da programação orientada a objetos, a capacidade de proteger os dados e controlar o acesso a eles é fundamental para criar sistemas robustos e confiáveis. O encapsulamento é um dos pilares desse paradigma, permitindo que os desenvolvedores escondam a complexidade interna das classes e exponham apenas o que é necessário para o uso externo. Isso não apenas promove a integridade dos dados, mas também facilita a manutenção e a evolução do software ao longo do tempo. Neste capítulo, exploraremos em profundidade o conceito de encapsulamento, seus benefícios, como implementá-lo em Java utilizando modificadores de acesso e métodos *getter* e *setter*, além de aprender como representá-lo visualmente em diagramas UML. Com exemplos claros e detalhados, você compreenderá como aplicar o encapsulamento para criar códigos mais seguros, limpos e eficientes.

6.1.1 O que é Encapsulamento?

Encapsulamento é o princípio de ocultar os detalhes internos de uma classe e expor apenas o necessário através de interfaces públicas. Em outras palavras, trata-se de agrupar dados (atributos) e métodos que operam sobre esses dados dentro de uma única unidade, a classe, e restringir o acesso direto a alguns dos componentes desse objeto.

6.1.2 Exemplo no Mundo Real

Imagine um carro moderno. Como usuário, você interage com o volante, o acelerador e os freios sem precisar entender como o motor, a transmissão ou os sistemas eletrônicos internos funcionam. O carro encapsula toda essa complexidade, proporcionando uma interface simples para o usuário.

Exemplo de Código:**Código_6.1:** Classe ContaBancaria em Java.

```
1 public class ContaBancaria {
2     private double saldo; // Atributo privado
3
4     // Método público para depositar dinheiro
5     public void depositar(double valor) {
6         if(valor > 0) {
7             saldo += valor;
8             System.out.println("Depósito realizado. Saldo atual: " + saldo);
9         } else {
10            System.out.println("Valor de depósito inválido.");
11        }
12    }
13
14    // Método público para sacar dinheiro
15    public void sacar(double valor) {
16        if(valor > 0 && valor <= saldo) {
17            saldo -= valor;
18            System.out.println("Saque realizado. Saldo atual: " + saldo);
19        } else {
20            System.out.println("Valor de saque inválido ou saldo insuficiente.");
21        }
22    }
23
24    // Método público para consultar o saldo
25    public double getSaldo() {
26        return saldo;
27    }
28 }
```

Neste exemplo, o atributo `saldo` é privado e só pode ser modificado através dos métodos públicos `depositar` e `sacar`. Isso garante que o saldo nunca seja alterado de forma inadequada.

6.1.3 Por que o Encapsulamento é Importante?

- **Proteção dos Dados:** Evita que dados sensíveis sejam alterados diretamente de fora da classe, garantindo a integridade das informações.

- **Facilidade de Manutenção:** Alterações internas na classe não afetam outras partes do sistema, desde que a interface pública permaneça inalterada.
- **Reutilização de Código:** Classes bem encapsuladas são mais fáceis de serem reutilizadas em diferentes contextos.
- **Redução de Acoplamento:** Minimiza a dependência entre diferentes partes do sistema, tornando-o mais modular e flexível.

6.1.4 Benefícios Práticos:

- **Segurança:** Dados críticos são protegidos contra alterações não autorizadas.
- **Flexibilidade:** A implementação interna pode ser alterada sem impactar os consumidores da classe.
- **Clareza:** Interfaces públicas claras facilitam o entendimento e o uso correto das classes.
- **Manutenção Simplificada:** Bugs e problemas podem ser isolados e corrigidos mais facilmente.

Exemplo Prático: Considere uma classe Aluno com atributos como nome e nota. Sem encapsulamento, qualquer parte do programa poderia alterar diretamente a nota de um aluno, possivelmente resultando em inconsistências. Com encapsulamento, a nota só pode ser alterada através de métodos que verificam se o novo valor é válido.

Código_6.2: Classe Aluno em Java.

```
1 public class Aluno {
2     private String nome;
3     private double nota;
4
5     public Aluno(String nome) {
6         this.nome = nome;
7         this.nota = 0.0;
8     }
9
10    public String getNome() {
11        return nome;
12    }
13
14    public double getNota() {
15        return nota;
16    }
17
18    public void setNota(double nota) {
19        if(nota >= 0 && nota <= 10) {
20            this.nota = nota;
21            System.out.println("Nota atualizada para: " + this.nota);
22        } else {
23            System.out.println("Nota inválida. Deve ser entre 0 e 10.");
24        }
25    }
26 }
```

6.2 Implementação do Encapsulamento em Java



O encapsulamento é um dos pilares fundamentais da programação orientada a objetos e se refere à ideia de esconder os detalhes internos de implementação de uma classe, permitindo o acesso a esses detalhes apenas por meio de métodos públicos específicos. Em Java, isso é alcançado utilizando modificadores de acesso como *private*, *protected* e *public*, que determinam a visibilidade dos membros de uma classe (variáveis e métodos). Ao tornar as

variáveis de uma classe privadas, garantimos que elas não possam ser acessadas diretamente de fora da classe, forçando o uso de métodos públicos (*getters* e *setters*) para ler ou modificar seus valores. Isso promove maior controle sobre como os dados são manipulados, permitindo, por exemplo, realizar validações ou ajustes antes de alterar o estado de um objeto.

6.2.1 Modificadores de Acesso

Java oferece quatro níveis de visibilidade para membros de uma classe (atributos e métodos), controlando como eles podem ser acessados de outras partes do programa.

1. **private**: O membro só pode ser acessado dentro da própria classe.
2. **default** (sem modificador): O membro pode ser acessado por qualquer classe no mesmo pacote.
3. **protected**: O membro pode ser acessado por classes no mesmo pacote e por subclasses em outros pacotes.
4. **public**: O membro pode ser acessado de qualquer lugar.

Tabela de Visibilidade:

Tabela_6.1: Tabela com os tipos de modificadores

Modificador	Classe	Pacote	Subclasse	Mundo
private	Sim	Não	Não	Não
default	Sim	Sim	Não	Não
protected	Sim	Sim	Sim	Não
public	Sim	Sim	Sim	Sim

Exemplo de Uso dos Modificadores:**Código_6.3:** Classe Pessoa em Java.

```
1 public class Pessoa {  
2     private String nome; // Só pode ser acessado dentro da classe Pessoa  
3     protected int idade; // Pode ser acessado por subclasses e classes do mesmo pacote  
4     String endereco; // default: acessível por classes do mesmo pacote  
5     public String telefone; // Acessível de qualquer lugar  
6  
7     // Métodos públicos para acessar e modificar o atributo privado 'nome'  
8     public String getNome() {  
9         return nome;  
10    }  
11  
12    public void setNome(String nome) {  
13        this.nome = nome;  
14    }  
15 }
```

6.2.2 Métodos Getter e Setter

Os métodos *getter* e *setter* são utilizados para acessar e modificar os atributos privados de uma classe de forma controlada.

- **Getter:** Método que retorna o valor de um atributo.
- **Setter:** Método que define ou atualiza o valor de um atributo, podendo incluir validações.

Exemplo de Implementação:**Código_6.4:** Classe Produto em Java.

```
1  public class Produto {  
2      private String nome;  
3      private double preco;  
4      // Getter para 'nome'  
5      public String getNome() {  
6          return nome;  
7      }  
8      // Setter para 'nome'  
9      public void setNome(String nome) {  
10         if(nome != null && !nome.isEmpty()) {  
11             this.nome = nome;  
12         } else {  
13             System.out.println("Nome inválido.");  
14         }  
15     }  
16     // Getter para 'preco'  
17     public double getPreco() {  
18         return preco;  
19     }  
20     // Setter para 'preco'  
21     public void setPreco(double preco) {  
22         if(preco >= 0) {  
23             this.preco = preco;  
24         } else {  
25             System.out.println("Preço não pode ser negativo.");  
26         }  
27     }  
28 }
```

Neste exemplo, os métodos *setter* garantem que os valores atribuídos aos atributos *nome* e *preco* sejam válidos, prevenindo erros e inconsistências no sistema.

6.2.3 Boas Práticas ao Utilizar *Getters* e *Setters*

- **Validação:** Sempre que possível, valide os dados antes de atribuí-los aos atributos.
- **Imutabilidade:** Se um atributo não deve ser modificado após a criação do objeto, forneça apenas um *getter* e não um *setter*.
- **Nomes Significativos:** Use nomes claros e descritivos para métodos, seguindo a convenção *getAtributo* e *setAtributo*.
- **Minimizar a Exposição:** Evite expor desnecessariamente muitos *getters* e *setters*, mantendo a classe o mais encapsulada possível.

Exemplo de Classe com Encapsulamento Completo:**Código_6.5:** Classe Livro em Java.

```
1 public class Livro {
2     private String titulo;
3     private String autor;
4     private int numeroPaginas;
5     private double preco;
6
7     // Construtor
8     public Livro(String titulo, String autor, int numeroPaginas, double preco) {
9         this.titulo = titulo;
10        this.autor = autor;
11        setNumeroPaginas(numeroPaginas);
12        setPreco(preco);
13    }
14
15    // Getters e Setters com validações
16    public String getTitulo() {
17        return titulo;
18    }
19
20    public void setTitulo(String titulo) {
21        if(titulo != null && !titulo.isEmpty()) {
22            this.titulo = titulo;
23        } else {
24            System.out.println("Título inválido.");
25        }
26    }
27 }
```

```
28     public String getAutor() {
29         return autor;
30     }
31
32     public void setAutor(String autor) {
33         if(autor != null && !autor.isEmpty()) {
34             this.autor = autor;
35         } else {
36             System.out.println("Autor inválido.");
37         }
38     }
39
40     public int getNumeroPaginas() {
41         return numeroPaginas;
42     }
43
44     public void setNumeroPaginas(int numeroPaginas) {
45         if(numeroPaginas > 0) {
46             this.numeroPaginas = numeroPaginas;
47         } else {
48             System.out.println("Número de páginas deve ser positivo.");
49         }
50     }
```

```
51
52     public double getPreco() {
53         return preco;
54     }
55
56     public void setPreco(double preco) {
57         if(preco >= 0) {
58             this.preco = preco;
59         } else {
60             System.out.println("Preço não pode ser negativo.");
61         }
62     }
63 }
```

6.3 Representação do Encapsulamento em UML



O diagrama de classes UML é uma ferramenta poderosa para visualizar o encapsulamento em um sistema orientado a objetos. A notação UML permite representar claramente a visibilidade dos atributos e métodos, facilitando a compreensão das relações entre as classes.

No diagrama de classe UML, o encapsulamento é representado de maneira visual, destacando os elementos que são privados ou públicos dentro de uma classe. A classe é representada por um retângulo, que é dividido em três seções: o nome da classe no topo, os atributos (ou variáveis) no meio e os métodos (ou operações) na parte inferior. Para ilustrar o conceito de encapsulamento, os atributos e métodos privados são indicados com o modificador de acesso -, enquanto os públicos são indicados com o +. Isso ajuda a visualizar claramente quais partes de uma classe podem ser acessadas diretamente e quais precisam ser manipuladas por meio de métodos controlados.

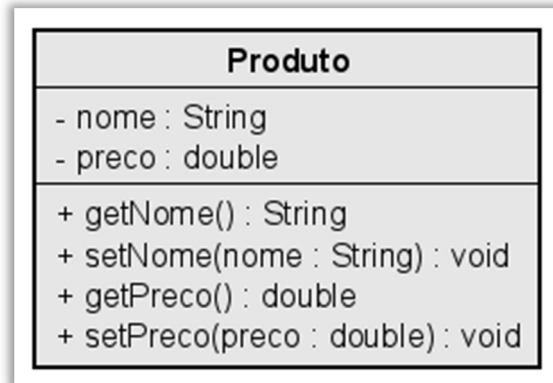
Por exemplo, em um diagrama UML de uma classe `ContaBancaria`, os atributos como `saldo` seriam marcados como privados, usando o sinal -, enquanto os métodos `depositar` e `retirar`, que manipulam o saldo, seriam públicos, representados pelo sinal +. Isso demonstra como o encapsulamento oculta o estado interno da classe (o saldo) e fornece um meio controlado de interação com esse estado, através dos métodos públicos. O diagrama de classe, assim, fornece uma visão clara de como a proteção e o controle de acesso são estruturados em um sistema orientado a objetos.

6.3.1 Notação de Visibilidade em UML

- **+ (Público)**: Indica que o membro é acessível de qualquer lugar.
- **- (Privado)**: Indica que o membro é acessível apenas dentro da própria classe.
- **# (Protegido)**: Indica que o membro é acessível dentro da classe e por subclasses.
- **~ (Package-Private)**: Indica que o membro é acessível por qualquer classe dentro do mesmo pacote.

Exemplo de Diagrama UML para a Classe Produto:

Imagem_6.1: Diagrama UML de Classe representando a classe Animal.



Neste diagrama:

- Os atributos *nome* e *preco* são privados (-), indicando que só podem ser acessados dentro da classe Produto.
- Os métodos *getNome*, *setNome*, *getPreco* e *setPreco* são públicos (+), permitindo acesso externo controlado aos atributos.

6.3.2 Diagrama UML com Encapsulamento em uma Hierarquia de Classes

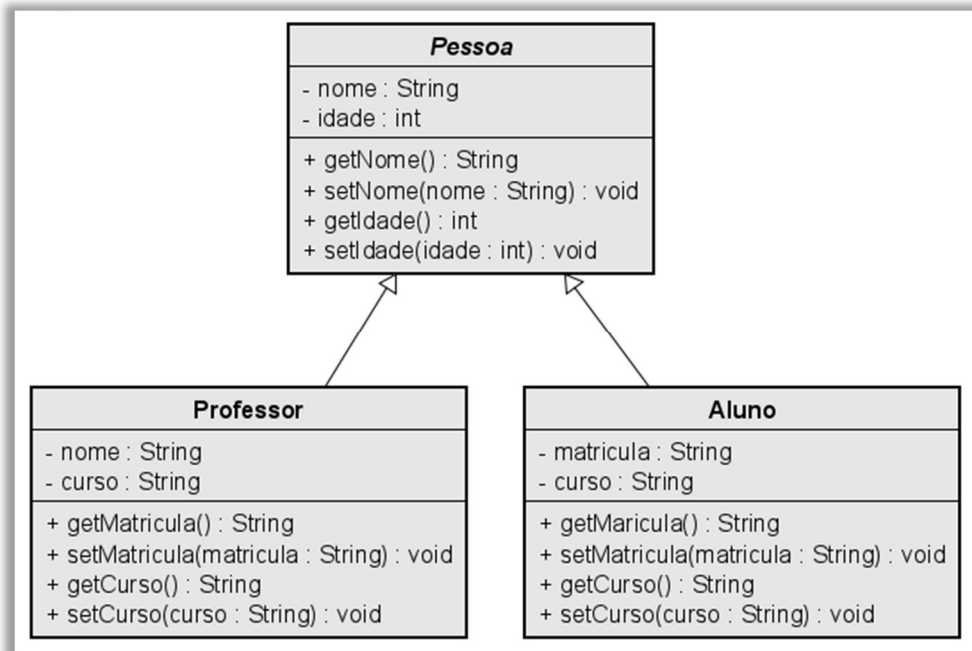
Vamos considerar uma hierarquia simples de classes para ilustrar o encapsulamento:

Classes:

- Pessoa (abstrata)
- Aluno
- Professor

Diagrama UML:

Imagem_6.2: Diagrama UML de Classe representando as classes Pessoa, Aluno e Professor.

**Descrição:**

- Pessoa é uma classe abstrata com atributos privados nome e idade, e métodos públicos para acessar e modificar esses atributos.
- Aluno e Professor herdam de Pessoa e adicionam seus próprios atributos e métodos encapsulados.
- A herança é representada por uma seta com triangulação oca apontando de Aluno e Professor para Pessoa.

Interpretação: Cada classe possui seus próprios atributos privados, garantindo que os dados sejam protegidos. Os métodos públicos (*getters* e *setters*)

permitem o acesso controlado a esses atributos, mantendo a integridade dos dados e promovendo o encapsulamento.

6.4 Exemplos Práticos



Sistema de Agenda de Contatos: A classe Contato pode usar encapsulamento para proteger informações como número de telefone e e-mail, permitindo acesso controlado.

Sistema de Pedidos de Restaurante: Uma classe Pedido pode encapsular os detalhes de cálculo de valor total e lista de itens, expondo apenas métodos públicos para interagir com esses dados.

Resumo do Capítulo



- **Encapsulamento:** Princípio fundamental da Orientação a Objetos que consiste em ocultar os detalhes internos de uma classe e expor apenas o necessário através de interfaces públicas.
- **Modificadores de Acesso:** Ferramentas fornecidas pelo Java (private, protected, public, e default) para controlar a visibilidade dos membros de uma classe.
- **Getters e Setters:** Métodos públicos utilizados para acessar e modificar atributos privados, permitindo validações e garantindo a integridade dos dados.
- **Benefícios do Encapsulamento:**
 - Proteção dos dados contra alterações não autorizadas.
 - Facilitação da manutenção e evolução do software.
 - Reutilização eficiente de código.
 - Redução do acoplamento entre diferentes partes do sistema.
- **Representação UML:** Utilização de símbolos de visibilidade (+, -, #, ~) para indicar o nível de acesso dos atributos e métodos, facilitando a visualização e o entendimento das classes e suas interações.

Ao dominar o encapsulamento, você está construindo uma base sólida para criar sistemas seguros, fáceis de manter e escaláveis. Este capítulo destacou como aplicar esses conceitos em Java e como representá-los visualmente, preparando-o para avançar nos próximos pilares da Orientação a Objetos.

Exercícios



1. **Definição de Encapsulamento:** Em suas próprias palavras, explique o que é encapsulamento e por que ele é essencial no desenvolvimento de software orientado a objetos.
2. **Modificadores de Acesso:** Dada a classe abaixo, identifique quais membros estão corretamente encapsulados e quais não estão. Justifique sua resposta.

Código_6.6: Classe Carro em Java.

```
1  public class Carro {  
2      public String modelo;  
3      private double preco;  
4      protected int ano;  
5      String cor;  
6  
7      public double getPreco() {  
8          return preco;  
9      }  
10  
11     public void setPreco(double preco) {  
12         if(preco > 0) {  
13             this.preco = preco;  
14         }  
15     }  
16 }
```

3. **Criação de uma Classe Encapsulada:** Crie uma classe Funcionario com os seguintes atributos privados: nome (String), salario (double) e departamento (String). Implemente métodos *getter* e *setter* para cada atributo, garantindo que:
 - O salário não possa ser negativo.

- O nome e o departamento não possam ser vazios ou nulos.
- 4. Análise de Código:** Observe o código abaixo e identifique onde o encapsulamento está sendo aplicado. Explique como ele protege os dados internos da classe.

Código_6.7: Classe Telefone em Java.

```
1 public class Telefone {
2     private String numero;
3     private String modelo;
4
5     public Telefone(String numero, String modelo) {
6         this.numero = numero;
7         this.modelo = modelo;
8     }
9
10    public String getNumero() {
11        return numero;
12    }
13
14    @ public void setNumero(String numero) {
15        if(numero.matches(regex: "\\d{10}")) {
16            this.numero = numero;
17        } else {
18            System.out.println("Número inválido.");
19        }
20    }
21
22    public String getModelo() {
23        return modelo;
24    }
25
26    public void setModelo(String modelo) {
27        this.modelo = modelo;
28    }
29 }
```

5. Desafio Prático: Imagine que você está desenvolvendo um sistema para gerenciar uma biblioteca. Liste três classes que fariam parte do modelo e descreva quais atributos e métodos cada uma teria. Em seguida, represente essas classes em forma de diagrama UML simplificado, destacando o encapsulamento dos atributos.

Conclusão



Com este capítulo, você adquiriu uma compreensão sólida sobre o encapsulamento, aprendendo a protegê-lo em suas classes Java e a representá-lo em diagramas UML. Esta habilidade é crucial para desenvolver softwares bem estruturados e seguros. No próximo capítulo, aprofundaremos outro pilar essencial da Orientação a Objetos: a Herança.